

CSEB1122T: Computer Programming

(using 'C' Language)

Ram Singh
Assistant Professor
Department of Computer Science & Engineering
PUNJABI UNIVERSITY PATIALA

Syllabus Outlines

(Section-A)

- **Number systems:** Bit, byte, binary, decimal, Hexadecimal and octals. Conversion from one base number to another base number.
- **Binary arithmetic:** Addition, subtraction, and multiplication.
- **Introduction to computer language:** Machine language, assembly, high-level, assembler, compiler, interpreter.
- **Introduction to Operating Systems:** Batch O.S., Multiprogramming, Time-sharing, Real-time, Network and distributed O.S.
- **Introduction to 'C':** Concepts of procedure oriented programming, character set, identifiers, keywords, data-types, and storage classes.
- **Operators and Expressions:** Arithmetic, unary, logical, relational, assignment, conditional operators. Associativity and precedence of operators.
- **Control Structures:** if, else, else if, while, do-while, for loops, nested control structures, switch-case, break, continue and break statements.

(Section-B)

- **Arrays:** 1D, 2D, 3D, array and pointers, string reading/writing.
- **Functions:** Types, call-by-value, call-by-reference, recursion, structures.
- **File processing:** Opening and closing data files, simple writing and reading in unformatted data files.
- **Object-Oriented Concepts:** Comparison of 'C' and C++, structures of C++ program, basics I/O statements. Introduction to classes and objects, creating a class and object, accessing class members (private, public).
- **C++ fundamental concepts:** Encapsulation, function overloading, single level inheritance, polymorphism and friend functions.

Introduction: Computer Programming

- **Data**
 - Collection
 - Storage
 - Processing
 - Results, transmission, sharing etc.

Number Systems

- In routine, we use numbers to represent counting, sequences, series, weights, height, lengths, width, cost, price, quantity, amount, sum, averages etc.
- We use 0-9 digits for such purposes i.e. 0, 05, 50, 599, 9559, 5879.95, 987650.005789, 3.14 etc.
- These number contains 10 (0-9) digits. Prefix “Deca” is used to represent number 10, so is used in many more representations, i.e. decameter" (10 meters) or "decibel" (one-tenth of a bel), "decagon" .
- Numbers formed using 0-9 digits, called 10 base number and are also termed as decimal number, i.e. 0-9 are 10 digits, so these are 10 base numbers.

conti...

- So depending on how many numeric digits are required to form a figure, we can create any base number, i.e. base 5 number will contain 0,1,2,3,4, base 8: 0,1,2,3,4,5,6,7, base 10: 0-9, base 2: 0,1.
- Because independent numeric digits 0-9 are 10 in numbers, to form number 10, 11, 12,...other symbols are used to represent these numbers in digital format, such as

Count	Digit	Value
1	0	0
2	1	1
3	2	2
4	3	3
5	4	4
6	5	5
7	6	6
8	7	7
9	8	8
10	9	9
11	A	10
12	B	11
13	C	12
14	D	13
15	E	14
16	F	15

conti...

Decimal Numbers

Count	Digit	Value
1	0	0
2	1	1
3	2	2
4	3	3
5	4	4
6	5	5
7	6	6
8	7	7
9	8	8
10	9	9

Hexa-decimal Numbers

Octal Numbers

Count	Digit	Value
1	0	0
2	1	1
3	2	2
4	3	3
5	4	4
6	5	5
7	6	6
8	7	7

Count	Digit	Value
1	0	0
2	1	1
3	2	2
4	3	3
5	4	4
6	5	5
7	6	6
8	7	7
9	8	8
10	9	9
11	A	10
12	B	11
13	C	12
14	D	13
15	E	14
16	F	15

Binary Numbers

Count	Digit	Value	Status
1	0	0	OFF
2	1	1	ON

conti...

- Positional weighted vs non-positional numbers:

1 ^{.....}	7 ¹⁰⁰⁰⁰⁰	9 ¹⁰⁰⁰⁰	8 ¹⁰⁰⁰	5 ¹⁰⁰	6 ¹⁰	4 ¹
--------------------	---------------------	--------------------	-------------------	------------------	-----------------	----------------

- Non-positional numbers:

I	II	III	IV	V	...	XXIV
---	----	-----	----	---	-----	------

i	ii	iii	iv	v	...	xxxv
---	----	-----	----	---	-----	------

conti...

Decimal Numbers

- 123 --- $(123)_{10}$
- 2450 --- $(2450)_{10}$
- 9876.75 --- $(9876.75)_{10}$
- 59998.00 --- $(59998.00)_{10}$

Octal Numbers

- 123 --- $(123)_8$
- 2450 --- $(2450)_8$
- 6776.75 --- $(6776.75)_8$
- 57677.00 --- $(57677.00)_8$

Hexa-decimal Numbers

- 123 --- $(123)_{16}$
- 2450 --- $(2450)_{16}$
- 98ABC --- $(98ABC)_{16}$
- DE159 --- $(DE159)_{16}$

Binary Numbers

- 0010 --- $(0010)_2$
- 1011 --- $(1011)_2$
- 01010 --- $(01010)_2$
- 100011 --- $(100011)_2$

Conversion to another base numbers

- To convert any number to another base number, we need to divide the number with that required number base. For example, to convert a base 10 number to base 2, we divide the 10 base number by 2 and obtain remainder less than 2 and continuously divided the quotients by base 2 till the last quotient is less than base or 0. The very first remainder numeric digit is to be called least significant bit (LSB) and the last digit is termed as most significant bit (MSB). This is applicable on all whole numbers of any base. Partial number after decimal point are multiplied with base and sum is calculated to place after decimal point. For example:-

Decimal to Binary conversion

- Convert decimal number $(85)_{10}$ to a binary number:

Divide decimal number by binary base 2	Quotient	Remainder	Binary
85/2	42	1	1 (LSB)
42/2	21	0	0
21/2	10	1	1
10/2	5	0	0
5/2	2	1	1
2/2	1	0	0
1/2	0	1	1 (MSB)

1 (MSB)	0	1	0	1	0	1 (LSB)
---------	---	---	---	---	---	---------

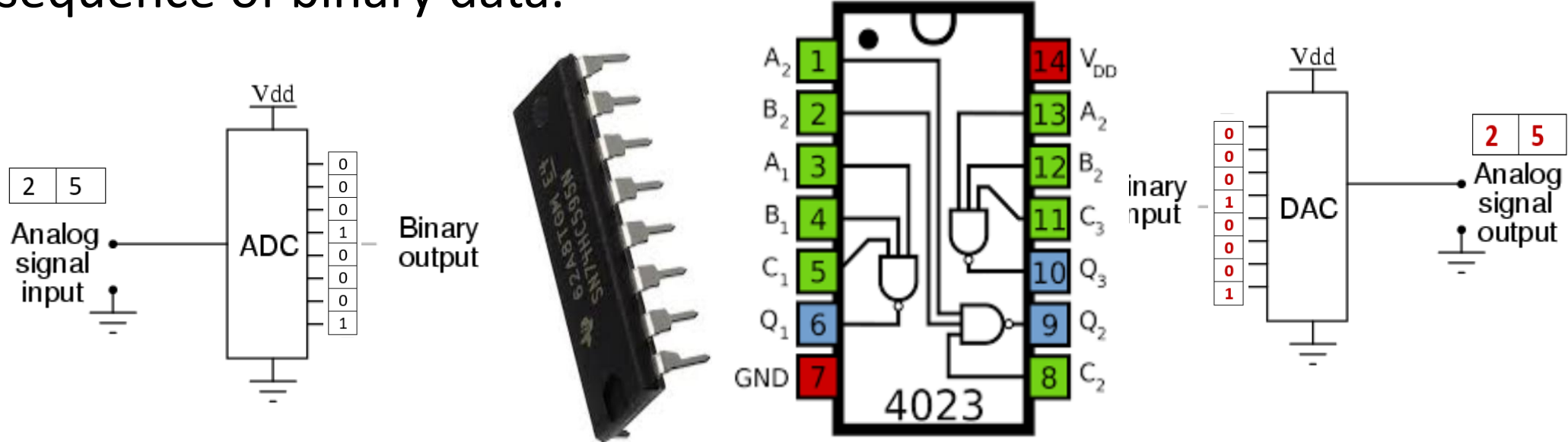
0	0	0	0	0	1	0	1
---	---	---	---	---	---	---	---

$$(85)_{10} = (1010101)_2$$

Count	Digit	Value	Binary
1	0	0	0
2	1	1	1
3	2	2	10
4	3	3	11
5	4	4	100
6	5	5	101
7	6	6	110
8	7	7	111
9	8	8	1000
10	9	9	1001
11	A	10	1010
12	B	11	1011
13	C	12	1100
14	D	13	1101
15	E	14	1110
16	F	15	1111

Bit, Byte, Binary

- A digital signal representing binary digits is a **pulse waveform that uses two discrete voltage levels to represent logic states 0 (LOW) and 1 (HIGH)**. The signal remains at one level for a specific time interval (bit length) and switches to the other level to represent a sequence of binary data.



conti...

- **Bit**: The term 'bit' is a word coined from two words, '**b**inary' and '**digit**' by concatenating '**bi**+'**it**', and is called '**bit**' to represent a digital signal value either **0** or **1**. It is a smallest unit of memory measurement to store data.
- **Byte**: A group of 8 bits, like **0 0 1 0 0 1 0 0 0** makes a byte.
- **Binary**: Number having base 2, i.e. uses symbols 0 and 1 are called binary numbers.
- Further these bits and bytes of binary are used to represent large data in terms of kilobytes (KB), megabytes (MB), gigabytes (GB), terabytes (TB) and so on.

Binary to Decimal conversion

- To convert a binary number to any other base number, multiply its position in the binary number counting first starting from 0 to right side in increasing order and add sum of all terms to obtain a converted number. For example:
- Convert previous obtained binary number **(1010101)₂** to decimal base 10 number

1 ⁶	0 ⁵	1 ⁴	0 ³	1 ²	0 ¹	1 ⁰
(MSB)						(LSB)
1×2 ⁶	0×2 ⁵	1×2 ⁴	0×2 ³	1×2 ²	0×2 ¹	1×2 ⁰
↓	↓	↓	↓	↓	↓	↓
64	0	16	0	4	0	1

Conversion of fractional numbers

- Decimal to binary:
 - Convert $(0.75)_{10}$ to binary number

Decimal	Binary	Fractional
0.75×2	1.	50 (MSB)
$.50 \times 2$	1.	00 (LSB)

- Convert $(0.11)_2$ to decimal

$$\frac{1}{2} + \frac{1}{4} = 0.50 + 0.25 = 0.75$$

Binary Arithmetic

Addition of Binary Numbers

Input-X	Input-Y	OUTPUT	Carry bit
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

with carry 1 to the
next significant place

0	3	5
0	2	4
0	5	9

	1	1
0	8	5
0	5	7
1	3	2

Example: 1 1 1 1

0	1	0	1	1
0	0	1	1	1
1	0	0	1	0

Binary notations

Hexa

Decimal

0
1
2
3
4
5
6
7
8
9

Octal

0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
1	1	0
1	1	1

0
1
2
3
4
5
6
7
8
9
A
B
C
D
E
F

0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1
1	0	1	0
1	0	1	1
1	1	0	0
1	1	1	0
1	1	1	1

Binary

0
1

X	Y
0	0
0	1
1	0
1	1

conti...

Subtraction of Binary Numbers

Binary addition and subtraction are performed using complements, primarily 2's complement, to simplify subtraction into an addition problem. For **addition**, simply add the binary numbers. For **subtraction**, find the 2's complement of the subtrahend (the number being subtracted), add this to the minuend (the number being subtracted from), and then handle any carry bit accordingly.

conti...

Binary subtraction using 2's complement

- **Process:** Subtracting a number is the same as adding its 2's complement.

1. Find the 2's complement of the subtrahend. This involves inverting all the bits (1's complement) and adding 1.
2. Add this 2's complement to the minuend.
3. **If a carry bit is generated**, discard it for a positive result.
4. **If no carry bit is generated**, the result is negative. Take the 2's complement of the result and place a negative sign in front.

Binary Subtraction

- Subtraction by additive method

$$(14)_{10} - (6)_{10} = (14)_{10} + (-6)_{10} = (8)_{10}$$

Take 2's complement of -6

0	0	0	0	0	1	1	0
1	1	1	1	1	0	0	1
							1
1	1	1	1	1	0	1	0

Add

C	1	1	1	1	1	1		
A	0	0	0	0	1	1	1	0
B	1	1	1	1	1	0	1	0
+	0	0	0	0	1	0	0	0

Ignore overflow bit-1

conti...

Subtract $(-5)_{10} - (-7)_{10} = (2)_{10}$ (using 1's complement method)

A = -5 and B = -(-7) = +7

Amplitude of 5 in binary is 0101 and 7 is 0111

Since -5 is negative, its 1's complement is

0	1	0	1
1	0	1	0

Since 7 is +ve, it is equal to binary

0	1	1	1
---	---	---	---

Add

C		1	1	1	1
A		1	0	1	0
B		0	1	1	1

Result

+	1	0	0	0	1
	1				
	0	0	0	1	0

conti...

- Using 2's complement method

- Take 2's complement of subtrahend and add it to the minuend by discarding final carry bit in the result.

-5 in 2's complement is:

0	1	0	1
1	0	1	0
		+	1
1	0	1	1

7 is already +ve number

0	1	1	1
---	---	---	---

Add 2's complement

	1	0	1	1
	0	1	1	1
1	0	0	1	0

Binary multiplication

Binary multiplication is similar to the multiplication of decimal numbers.

Multiplicand	Multiplier	Product
0	0	$0 \times 0 = 0$
0	1	$0 \times 1 = 0$
1	0	$1 \times 0 = 0$
1	1	$1 \times 1 = 1$

$$\begin{array}{r} \\ \\ \\ \\ + \\ \hline 1 \end{array}$$

$$(1111)_2 \times (1010)_2 = (10010110)_2$$

conti...

- Computer Method of Binary multiplication:

Suppose we want to multiply binary numbers $(1101)_2$ and $(1011)_2$
 $= (13 \times 11) = 143$

Place multiplicand in register MQ (multiplicand and quotient):	1	1	0	1	0	0	0	0
Place multiplier contents in register M :	0	0	0	0	1	0	1	1

Step-1: Shift contents of register MQ 1-bit left.	1	1	0	1	0	0	0	0
New contents in MQ		1	0	1	0	0	0	0
If outgoing bit is 1, add contents of M into MQ		0	0	0	0	1	0	1
Partial sum of step 1 is stored in register MQ		1	0	1	0	1	0	1

Step-2: Shift contents of register MQ 1-bit left.	1	0	1	0	1	0	1	1
New contents in MQ		0	1	0	1	0	1	1
As outgoing bit is 1, add contents of M into MQ		0	0	0	0	1	0	1
Partial sum of step 2 is stored in register MQ		0	1	1	0	0	0	1

conti...

Step-3: Shift contents of register MQ 1-bit left.		0	1	1	0	0	0	0	1	
	New contents in MQ		1	1	0	0	0	0	1	0
	As outgoing bit is 0, add 0 into MQ		0	0	0	0	0	0	0	0
	Partial sum of step 3 is stored in register MQ		1	1	0	0	0	0	1	0

Step-4: Shift contents of register MQ 1-bit left.		1	1	0	0	0	0	1	0	
	New contents in MQ		1	0	0	0	0	1	0	0
	As outgoing bit is 1, add contents of M into MQ		0	0	0	0	1	0	1	1
	Final product of step 4 is stored in register MQ		1	0	0	0	1	1	1	1

1^7	0^6	0^5	0^4	1^3	1^2	1^1	1^0
-------	-------	-------	-------	-------	-------	-------	-------

1×2^7	0×2^6	0×2^5	0×2^4	1×2^3	1×2^2	1×2^1	1×2^0
----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------

128	0	0	0	8	4	2	1
-----	---	---	---	---	---	---	---

128	0	0	0	8	4	2	1	=143
-----	---	---	---	---	---	---	---	------

Introduction to Computer Languages

Machine Language (ML):

- ML is the lowest-level programming language, directly understood and executed by a computer's Central Processing Unit (CPU).
- It consists of binary instructions, represented as sequences of 0s and 1s, specific to a particular CPU architecture.
- Writing programs directly in machine language is tedious and error-prone, time consuming and highly difficult for humans, but not impossible.

conti...

Assembly Language:

- A low-level programming language that uses mnemonics (symbolic codes) to represent machine language instructions.
- Provides a more human-readable alternative to machine language, but still requires detailed knowledge of the CPU architecture.
- Each assembly language is specific to a particular computer architecture (e.g., x86, ARM).

conti...

High-Level Language (HLL):

- Programming languages designed to be more abstract and human-readable, using syntax closer to natural language.
- Examples include C, C⁺⁺ , Pascal, BASIC, Python, Java, JavaScript etc.
- HLL are generally machine-independent, meaning code written in a high-level language can be run on different computer architectures with appropriate translation.

conti...

Assembler:

- It is a software tool for an assembly language processing that translates assembly language code into machine language.
- It converts the symbolic instructions and data representations of assembly language into their binary equivalents that the CPU can execute.

conti...

Compiler:

- It is a system software for programming language processing to compile and translates an entire program written in a high-level language, like C/C++/Pascal, into machine language (or an intermediate code) before execution.
- The compilation process typically involves multiple stages, including lexical analysis, parsing, semantic analysis, and code generation.
- The output of a compiler is an executable file that can be run independently without the compiler.

conti...

Interpreter:

- A language processor that translates and executes high-level language code line by line, or statement by statement, at runtime.
- Unlike a compiler, an interpreter does not produce a separate executable file.
- The interpreter must be present during each execution of the program. For web technologies like HTML, Python and other scripting languages, an interpreter is used.

Introductino to Operating Systems

Batch operating system

- **Batch O.S.** processes groups of jobs with similar needs into batches and processes them one after another without user interaction.
- **Example:** Processing payroll or billing tasks that are repetitive and don't require immediate input.
- **Key Feature:** Lack of user interaction during job processing.
- These type of systems are absolete nowadays.

conti...

Multiprogramming operating system

- These O.S. keeps several programs in memory simultaneously. When one program is waiting for I/O, the CPU can switch to another program to keep it busy.
- **Example:** Running multiple applications like a word processor, web browser, and music player on a single computer.
- **Key Feature:** Maximizes CPU utilization by switching between programs.

conti...

Time-sharing operating system

- A logical extension of multiprogramming that allows multiple users connected with the centralized servers through intranet and internet channels to interact with the system at the same time by giving each a small time slice of the CPU's processing time.
- **Example:** Modern desktop operating systems like Windows, macOS, and Unix/Linux, where many users can work simultaneously.
- **Key Feature:** Provides an interactive environment for multiple users by rapidly switching between tasks.

conti...

Real-time operating system (RTOS)

- RTOS are designed to handle tasks that must be completed within a specific time frame. These system are highly dedicated for time-critical task handling and responsive to every real-time critical data handling, receiving, processing, sending, storing and alerting the users.
- Example: Systems used in medical devices, industrial control, or automotive systems that require precise timing. Used in satellite data communications, air-line traffic control, weather forecasting, scientific industrial control such as nuclear reactors controlling etc.
- Key Feature: Focuses on task completion deadlines rather than just multitasking efficiency.

conti...

Network and distributed operating system

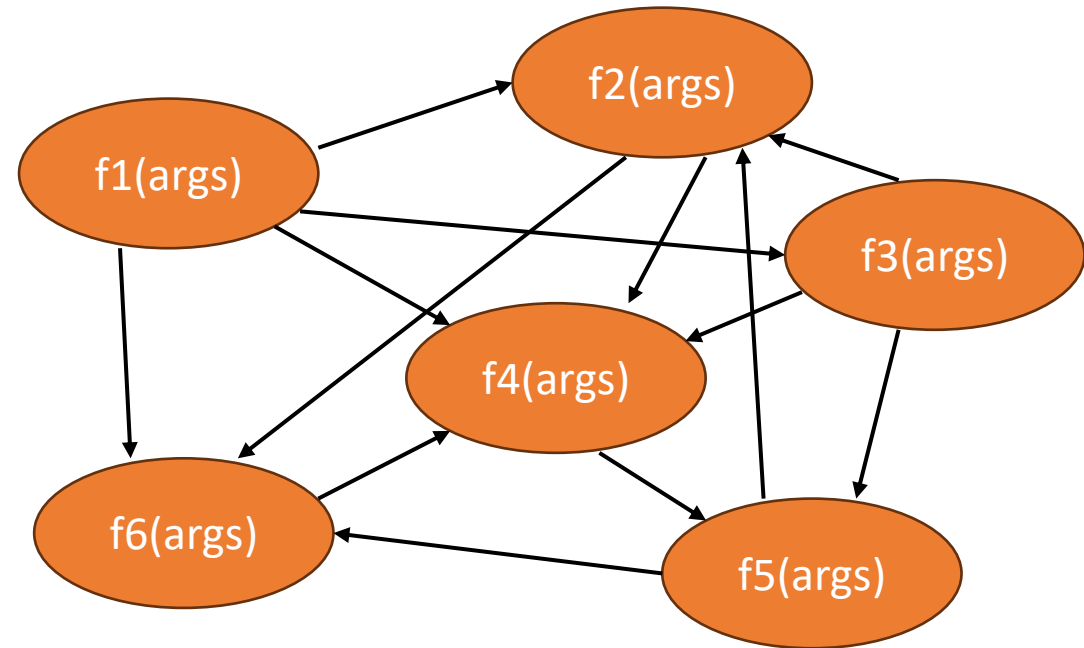
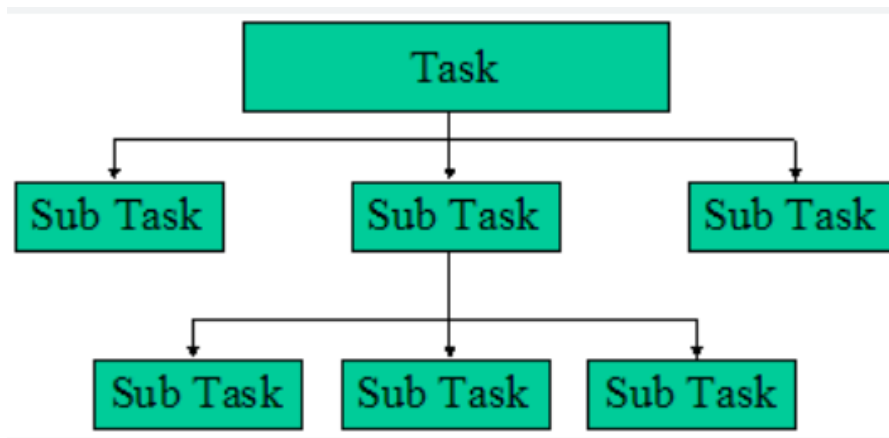
- Networked OS: :
 - Manages network resources, allowing different computers to share files, printers, and other services.
 - Distributed OS: Manages resources across multiple computers as if they were a single system, often hiding the underlying complexity of the individual machines.
- Example: A network operating system could be Novell NetWare, while a distributed OS might be something like Amoeba.
- Key Feature: Enables communication and resource sharing across multiple computers.

Introduction to 'C' Language

- 'C' was developed at Bell Labs by **Dennis Ritchie (1941–2011), an American computer scientist**, in the early 1970s. C is a widely used and influential language known for its efficiency and portability. It has served as a basis for many other programming languages.
- 'C' is a high-level, compact and very powerful computer programming language.
- It follows top-down programming approach that means, a step-by-step procedure is used to design the structure of program to solve any given problem.

conti...

- Data and functions move freely, and functions can be used by passing data arguments from anywhere by one function to another.



Character set, identifiers, keywords, data-types, and storage classes in 'C'

The C programming language is built upon fundamental elements including a specific character set, rules for identifiers, reserved keywords, various data types, and storage class specifiers.

Character set:

- The C character set is the set of valid characters that can be used in source code. It includes:
- **Alphabets:** Uppercase letters (A-Z) and lowercase letters (a-z).
- **Digits:** Numerals from 0 to 9.
- **Special Characters:** A rich set of symbols used for operations, punctuation, and control, such as +, -, *, /, =, <, >, &, |, !, #, %, ;, :, ?, (,), [,], {, }, _, ' , " , \ , ~ , . , and ,.
- **Whitespace Characters:** Blank space, newline (\n), horizontal tab (\t), carriage return (\r), and form feed (\f).

conti...

Identifiers:

- Identifiers are user-defined names given to program elements such as variables, functions, arrays, structures, and labels. They must adhere to specific rules:
- The first character must be an **alphabet** (uppercase or lowercase) or an **underscore** (_).
- Subsequent characters can be **letters**, **digits**, or **underscores**.
- **Keywords cannot be used** as identifiers.
- **No special characters** (except the underscore) or **whitespace** are allowed within an identifier.
- Identifiers are **case-sensitive** (e.g., age and Age are different).
- Most modern compilers support identifiers up to at least 31 characters in length.

conti...

Keywords:

Keywords are predefined, reserved words that have special meanings to the C compiler and are used to define the language's syntax and structure. They are always written in lowercase.

The standard C language has 32 keywords (additional ones like **_Bool** or **_Alignas** were added in newer standards like C99 and C11):

Category	Keywords
Data Types	<code>char</code> , <code>int</code> , <code>float</code> , <code>double</code> , <code>void</code> , <code>short</code> , <code>long</code> , <code>signed</code> , <code>unsigned</code>
Control Flow	<code>if</code> , <code>else</code> , <code>switch</code> , <code>case</code> , <code>default</code> , <code>for</code> , <code>while</code> , <code>do</code> , <code>break</code> , <code>continue</code> , <code>goto</code>
Storage Classes	<code>auto</code> , <code>register</code> , <code>static</code> , <code>extern</code>
Other	<code>sizeof</code> , <code>return</code> , <code>typedef</code> , <code>struct</code> , <code>union</code> , <code>enum</code> , <code>const</code> , <code>volatile</code>

Data-types:

Data-types:

Data types specify the type of data a variable can store and how much memory it occupies. C has fundamental and derived data types:

- **Primary (Basic) Data Types:**

int	Used to stores whole numbers (integers).
char	Used to stores a single character/strings
float	Used to stores single-precision floating-point numbers (decimals).
double	Stores double-precision floating-point numbers, with roughly twice the precision of float .
void	Specifies that a function does not return a value or a NULL pointer to generic data.

- **Derived Data Types:**

- Arrays, pointers, structures (**struct**), and unions (**union**).

- **User-Defined Data Types:**

- Defined using **typedef** or **enum**.

Storage classes...

Storage classes:

Storage classes in C determine the **scope, visibility, and lifetime** of variables and functions.

- **auto**: The default storage class for local variables. Variables are created when the block is entered and destroyed when the block is exited.
- **register**: Suggests to the compiler that the variable should be stored in a CPU register for faster access (this is a suggestion the compiler may ignore).
- **static**: Preserves the value of a variable between function calls (maintains its value across the program's execution, even if its scope ends).
- **extern**: Declares a global variable or function that is defined in another file or later in the current file, allowing for shared variables across multiple files.

Operators and Expressions

- In C programming, operators are symbols or keywords that perform specific operations on operands, which are the data items or variables they act upon. An expression is a combination of operators, operands, and function calls that evaluates to a single value.

- **Types of Operators in C:**

Arithmetic Operators: Used to perform mathematical operations on data		Examples
+	Addition, unary plus (+ve)	$x = a + b$
-	Subtraction, unary minus (-ve)	$x = a - b$
*	Multiplication	$x = a * b$
/	Division	$x = a / b$
%	Modulus - reminder	$x = a \% b$

Relational Operators: Used to compare two operands/variable values and return a Boolean result as (True or 1, False/0)		Examples
==	Equality	$a == b$
!=	Not equal	$a != b$
>	Greater than	$a > b$
<	Less than	$a < b$
>=	Greater than or equal to	$a >= b$
<=	Less than or equal to	$a <= b$

conti...

Logical Operators: Used to combine two or more expressions for <u>boolean operations</u> .		Examples
&&	Logical AND	a && b
	Logical OR	a b
!	Logical NOT	<u>a !a</u>

Increment/Decrement Operators: Used to increase/decrease a variable value by one.		Examples
++	Increment value <u>by 1</u>	a++;
--	Decrement value by 1	a--;
Also used in prefix (++a/--a) or postfix (a++/a--) form pre OR post increment/decrement.		

Assignment Operators: Used to assign or store value written right side of = variable		Examples
=	Assigning/writing in memory	x= <u>a+b</u>
Compound/compact use with other operators		
+=	Assign after adding	x+=a
-+	Assign after subtracting	x/=ab
=	Assign after multiplication	x=b
/=	Assign after division	x/=b
%=	Assign after <u>modulu</u>	x%=b

conti...

Conditional (Ternary) Operator: It is a short-cut notation of if..else...

condition ? expression1 : expression2;

If condition finds true, expression is evaluated, else expression is evaluated.

```
int a=5, b=2, c=0;
```

```
C=(a==b) ? 10:5;
```

```
printf("C=%d",c)
```

conti...

Bitwise Operators: Used to perform operations directly on the bits of integer values.		Examples
&	Bitwise AND	$X = X \& 25$
	Bitwise OR	$X = X 25;$
^	Bitwise XOR	$X = X ^ 25$
~	Bitwise NOT	$X = \sim X;$
<<	Left shift	$X = X << 2;$
>>	Right shift	$X = X >> 2$

Operator	Example	Explanation
<< left shift	$X = X << 2;$	Before 0000 1111 X is 15 (8+4+2+1) After 0011 1100 X is 60 (32+16+8+4)
>> right shift	$X = X >> 2;$	Before 0000 1111 X is 15 (8+4+2+1) After 0000 0011 X is 3 (2+1)
& bit-wise AND	$X = X \& 28;$	0000 1111 & 0001 1010 = 0000 1010 15 28 10
bit-wise OR	$X = X 28;$	0000 1111 0001 1010 = 0001 1111 15 28 31
^ bit-wise XOR	$X = X ^ 28;$	0000 1111 ^ 0001 1010 = 0001 0101 15 28 21
~ bit inversion	$X = \sim X;$	Before 0000 1111 X is 15 (8+4+2+1) After 1111 0000 X is -2,147,483,633

- **Special Operators:**

sizeof(): Returns the size of a data type or variable in bytes.

& (address-of operator): Returns the memory address of a variable.

*** (dereference operator):** Accesses the value at a memory address (used with pointers).

, (comma operator): Evaluates multiple expressions from left to right, returning the value of the last expression.

Associativity and precedence of operators.

- Operator precedence determines which operators are evaluated first in an expression, while associativity determines the order of evaluation for operators with the same precedence. For example, multiplication has higher precedence than addition, so in the expression $2 + 3 * 5$, multiplication is performed first, resulting in 17. When operators have the same precedence, such as division and multiplication, associativity (typically left-to-right) determines the order, as in $6 * 3 / 2 = (6 * 3) / 2$.

Operator precedence

- **Definition:** - The priority of different operators in an expression. Operators with higher precedence are evaluated before those with lower precedence.
- **Example:** In $2+3*5$, the $*$ operator has higher precedence than $+$, so the multiplication is done first, making the expression $2+15$, which equals 17.
- **Parentheses:** Parentheses $()$ can be used to override default precedence and force a different order of operations. For example, in $(2+3)*5$, the addition is performed first, resulting in 25.

Operator associativity

- **Definition:** The order in which operators with the same precedence are evaluated, either from left to right or right to left.
- **Example:** The $*$ and $/$ operators typically have the same precedence and are evaluated left-to-right. In $40/4*5$, the expression is evaluated as $(40/4)*5$, which is $10*5=50$.
- **Right-to-left associativity:** Some operators, like the assignment operator ($=$), associate from right to left. For example, in $a=b=5$, the value 5 is assigned to b , and then the value of b is assigned to a , so both a and b become 5.

Control Structures

- Control structures in 'C' programming consists **if, else, else if, while, do-while, for** loops, nested control structures, **switch-case-default, break, continue** and **break** statements.

Arrays:

- An array is a type of data **structure or container** than can store **similar type/homogeneous types** of data elements into a single variable of array type. For example we can store **only integer type** or **character types** of data values in a array.

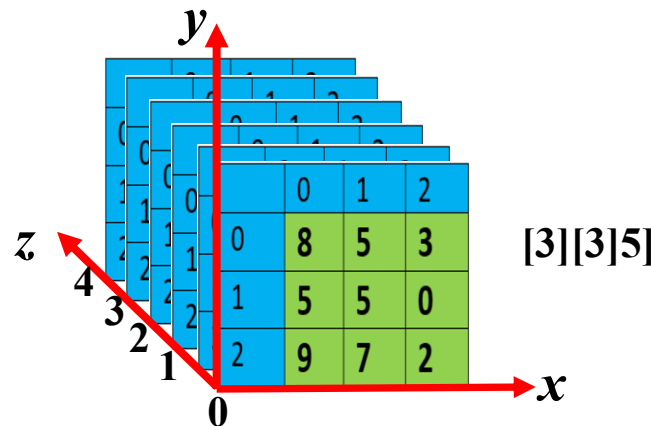
1-D Array (column major)

IND	Data
-----	------

0	5
1	8
2	0
3	8
4	10
5	8
6	7
7	5
8	6
9	3

1-D Array (Row major)

	ArrName									
Index	0	1	2	3	4	5	6	7	8	9
Data Values	2	5	5	4	2	1	9	0	12	10



2-D Array

	0	1	2
0	8	5	3
1	5	5	0
2	9	7	2

String reading/writing

- Format conversion **"%s"** can be used in **scanf()** for reading strings not containing white spaces: **scanf("%s", str)**
- **'&'** not required before str as it is a pointer.
- C string library provides **gets(str)** to read a string.
- It does not skip white spaces like **scanf()** does.
- But, **scanf()** and gets have no way to know the size of the array in advance.
- So potentially, they could attempt to store character past the array.
- **scanf()** can use **"%.ns"** conversion to avoid this problem.
- But **gets()** is dangerous.

Reading and writing strings

- Since both **scanf()** and **gets()** are potentially risky, character by character reading using **getchar()**, and similarly writing using **putchar()** are safer to use.
- To read or write a single character or strings character by character, use **%c** format specifier.
- For strings, use **%s** format specifier
- Example for reading a string:

```
char str[]="Punjabi University Patala";  
printf("%s",str);
```

String Functions

C provide rich set of string functions.

#include <string.h> to access these string manipulation functions.

strlen(s): returns length of the string.

strcpy(dest, src): copies one string into another.

strcat(dest, src): appends one string to another.

strcmp(first, second): return 1 if equal 0 otherwise.

Functions

- **Types**
 - Inbuilt (library) functions
 - User – defined functions
 - Non-parameterized functions
 - Parameterized functions
- call-by-value,
- call-by-reference
- Recursion/recursive functions

Conti...

- A **function** or **functions** are self-contained group of executable statements/expressions written in '**C**' to perform any specific task or operation on data.

Syntax:

```
<data_type function_Name(argument1, argument2, argumentN,...)
```

```
{
```

```
    expressions;
```

```
    ...
```

```
}
```

```
>
```

```
int calSum(int x, int y)
{
    int sum=0;
    sum=x+y;
    printf("Sum=%d", sum);
}
```

Types of functions

Basic four types of functions can be used in 'C' programming:

- 1. Without arguments, without returning a value.
- 2. Without arguments, with returning a value.
- 3. With arguments, without returning a value.
- 4. With arguments, with returning a value.

Conti...

- Declaration of function (function prototyping).
- Defining a function.
- Calling a function
 - Calling by passing values – called 'Call-by-value'
 - Calling by sending addresses of values – called 'Call by reference'

Recursion

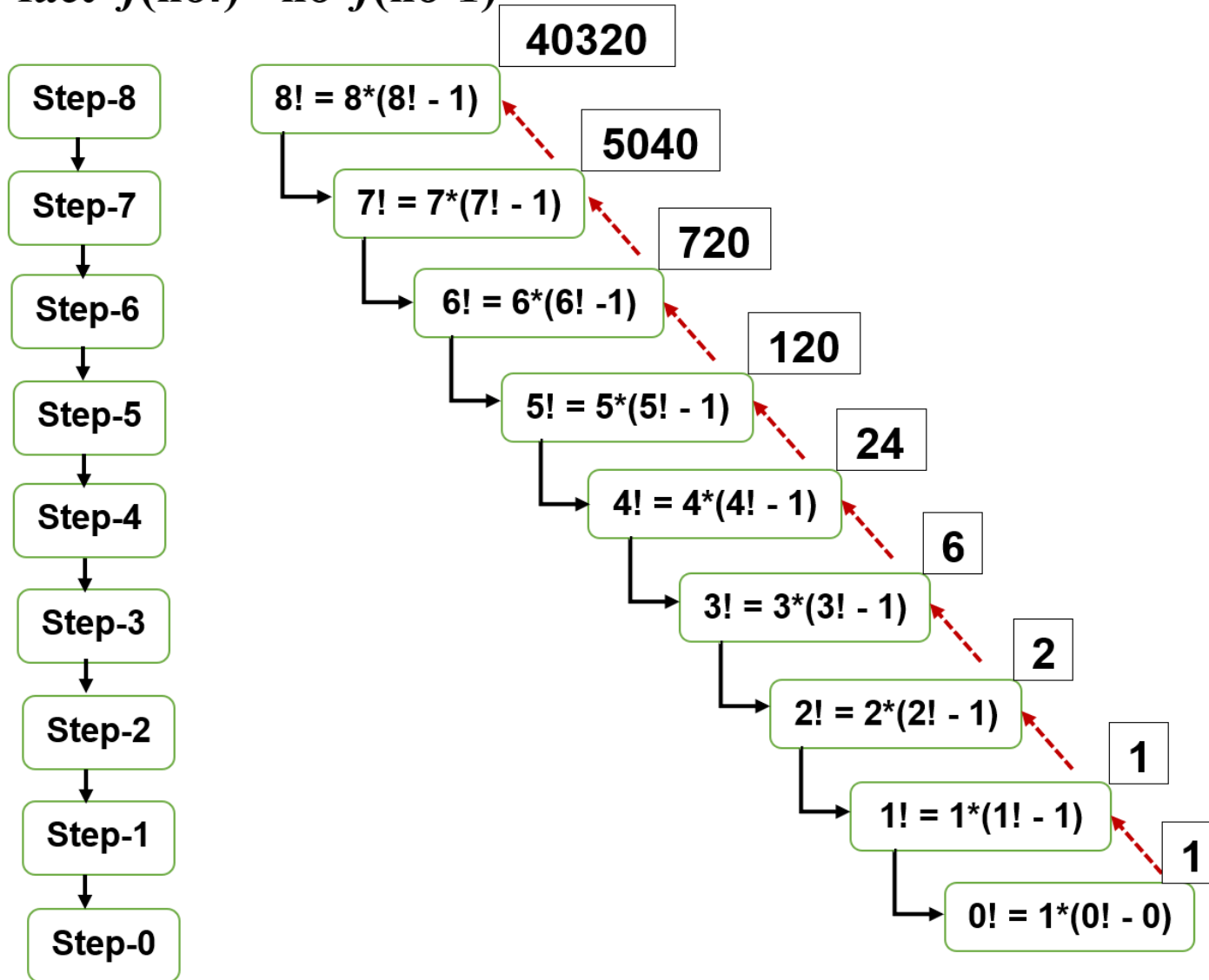
- Recursion is a technique used to **call a function by itself**, directly or indirectly within from its own body till a base condition met true.
- A function that implemented to exhibit its self-calling behaviour is known as a recursive function.
- This technique is used to solve complicated problems dividing into similar smaller subproblems to solve efficiently in simpler ways.

conti...

- **Base condition:** For a recursive function, a base condition is required to set that stops the recursion. Without a base condition, it would continuously or infinitely call itself that will lead to a memory stack overflow error. Base condition defines the simplest form of the problem to solve it directly without further recursion.
- **Recursive case:** In recursion, function call itself with modified parameters, and goes closer to the base condition by operating on a smaller or simpler form of the original problem.

$$\text{fact} = f(\text{no}!) = \text{no} * f(\text{no} - 1)$$

conti...



$$8! = 8 * 7 * 6 * 5 * 4 * 3 * 2 * 1 * 1 = 40320$$

Structure and union

- A structure is **a container** type variable that can store multiple data members of different variables of different (**heterogeneous data type**).
- It can be declared using a keyword **struct** and any user specific structure name.
- After that a structure variable can be created as instance of the structure that exhibit its all data members of specific data types.
- Each data-member of a structure holds data values on their individual memory locations.

conti...

- **Structure example:**

int a; // 4 bytes

char ch; //1 byte

float avgMarks; //4 bytes

float perMarks; //4 bytes

double factorial; //8 bytes

```
struct student{ int rollNo;  
                char studName[50];  
                char entryExam[30];  
                float avgMarks;  
                float perMarks;  
                char course[50];  
                char batchYear[4];  
                };
```

conti...

- How to use structure to write/store data elements of each data member of structure ?
- As we create variables of basic data types, we can create variables of a struct type structure, i.e.

```
int fact=1;
```

```
int main()  
{  
    struct student s;  
    s.rollNo=55;  
    printf("Hello world!\nRoll Number=%d", s.rollNo);  
    return 0;  
}
```

union

- **union** is a keyword, used in place of **struct** keyword to declare a structure type variable. As its name implies, it allows all data member variables of a **union** type structure to share a **single large memory address** equal to the large memory variable within a structure.
- In this way memory is not allocated to each data members in a **union** type structure like **struct** type structure.
- It saves a lots of memory depending upon the number of variables in a structure.
- To create a union type strcutue, use keyword **union** inplace of **struct**.

conti...

- Union type structure:

union equal to max. size address

```
union Student{  
    int rollNo;  
    char studName[50];  
    char entryExam[30];  
    float avgMarks;  
    float perMarks;  
    char course[50];  
    char batchYear[4];  
};
```

File processing

- A file is a user given named structure create to store programs and data permanently on secondary storage devices. These can be accessed for data writing, reading, processing, modifying, adding new data, deleting existing data etc.
- Every file is a extension from one single character to 3 or 4 character long. For example, a 'C' program has a extension as **.c**, a C++ program has a extension as **.cpp**. Other file extensions are **.doc**, **.ppt**, **.docx**, **.xls**, **xlsx**, **.txt**, **.dat**, **.jpg**, **.tif**, **tiff**, **.bmp**, **.png**, **.avi**, **.mvi**, **.mp**, **.mpeg**, **.jpeg** etc.

conti...

- Using 'C' programming, any type of data file is created using **FILE** type pointer variable to the memory.

```
int main()  
{  
    FILE *fptr;  
    fptr=fopen("D:\\studentsz.txt","w");  
    return 0;  
}
```

File Opening modes

Text Modes	Description	Binary Modes:	These modes are similar to the text modes but are specifically for handling binary files, which are treated as raw bytes without any character set translations.
"r" (read)	Opens an existing text file for reading. If the file does not exist, <code>fopen()</code> returns NULL.	" <u>rb</u> "	read binary
"w" (write)	Opens a text file for writing. If the file exists, its contents are truncated (erased). If the file does not exist, a new file is created.	" <u>wb</u> "	write binary
"a" (append)	Opens a text file for appending data to the end. If the file does not exist, a new file is created.	"ab"	append binary
"r+" (read and write)	Opens an existing text file for both reading and writing. The file must exist.	" <u>r+b</u> " or " <u>rb+</u> "	read and write binary
"w+" (write and read)	Opens a text file for both reading and writing. If the file exists, its contents are truncated. If the file does not exist, a new file is created.	" <u>w+b</u> " or " <u>wb+</u> "	write and read binary
"a+" (append and read)	Opens a text file for both reading and appending. If the file does not exist, a new file is created. Reading can occur from the beginning, but writing is always at the end.	" <u>a+b</u> " or " <u>ab+</u> "	append and read binary

File operations

- Open a new file for writing
- Open an existing file for reading, editing, modifying, rewriting etc.
- Rename an existing file with new file name.
- Copy a file.
- Delete a file
- Edit a file
- Close a file

Opening a New File

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    FILE *fptr;
    if (fp == NULL)
    {
        printf("Error opening file!\n");
        return 1;
    }
    fptr=fopen("D:\\studentsz.txt", "w");
    // Write to the file
    fprintf(fp, "Hello, this is a test file.\n");
    // Always close the file
    fclose(fp);
    printf("File created and written successfully.\n");
    return 0;
}
```

Writing a structure to a file

```
#include <stdio.h>
#include <stdlib.h>
struct Student {
    int roll;
    char name[50];
    float marks;
};

int main() {
    struct Student s = {1501101, "Ram Singh", 88.5};

    FILE *fp = fopen("student.txt", "w");
    if (fp == NULL) {
        printf("Error opening file!\n");
        return 1;
    }
    // Write structure fields as text
    fprintf(fp, "%d\n", s.roll);
    fprintf(fp, "%s\n", s.name);
    fprintf(fp, "%.2f\n", s.marks);
    fclose(fp);
    printf("Student data written to student.txt\n");

    return 0;
}
```

Writing structure data into a binary format

```
#include <stdio.h> //writing data into a binary format|
#include <stdlib.h>
struct Student {
    int roll;
    char name[50];
    float marks;
};

int main() {
    struct Student s = {101, "Ram Singh", 88.5};
    FILE *fp = fopen("student.dat", "wb"); // binary write
    if (fp == NULL) {
        printf("Error opening file!\n");
        return 1;
    }
    // Write the entire structure in binary
    fwrite(&s, sizeof(struct Student), 1, fp);
    fclose(fp);
    printf("Student data written to student.dat (binary)\n");
    return 0;
}
```

Object-Oriented Concepts using C++

Programming Paradigm:

- **C** is a procedural programming language, focusing on functions and sequential execution.
- **C++** is a multi-paradigm language, supporting both procedural and object-oriented programming (**OOP**). **OOP** features in **C++** include **classes**, **objects**, **inheritance**, **polymorphism**, and **encapsulation**.

Input/Output:

- **C** uses **printf()** and **scanf()** for standard output and input, respectively. **C++** employs **cout** and **cin** for output and input, often with stream manipulation features.

Object-Oriented Features:

- **C** does not support **OOP** concepts like classes, objects, and inheritance directly. **C++** fully incorporates these features, enabling the creation of reusable and modular code.

Memory Management:

- **C** uses functions like `malloc()`, `calloc()`, and `free()` for dynamic memory allocation and deallocation. **C++** utilizes the `new` and `delete` operators for the same purpose, often integrated with object construction and destruction.

Data and Functions:

- In **C**, data and functions are generally separate. In **C++**, data and functions can be encapsulated within classes, promoting data hiding and better organization.

Function Overloading:

- **C** does not support function overloading, meaning multiple functions cannot share the same name even with different parameter lists. **C++** allows function overloading, enhancing code readability and flexibility.

Error Handling:

- **C** typically relies on return codes or global error variables for error handling. **C++** provides more robust exception handling mechanisms using try-catch blocks.

Namespaces:

- **C** does not have namespaces. **C++** introduces namespaces to prevent name collisions in large projects or when using multiple libraries.

Difference in C and C++ syntax

'C' syntax

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int age;
    char name[30];
    printf("\nEnter your age:");
    scanf("%d", &age);
    printf("\nEnter your name:");
    scanf("%s", name);
    printf("\nMr. %s, you are %d year old", name, age);
    return 0;
}
```

'C++' syntax

```
#include <iostream>
using namespace std;

int main()
{
    int age;
    char name[30];
    cout<<"\nEnter your age:"<<endl;
    cin>>age;
    cout<<"\nEnter your name:"<<endl;
    cin>>name;
    cout<<"\nMr."<<name<<" you are "<<age<<" year old."<<endl;
    return 0;
}
```


Structure of a C++ program

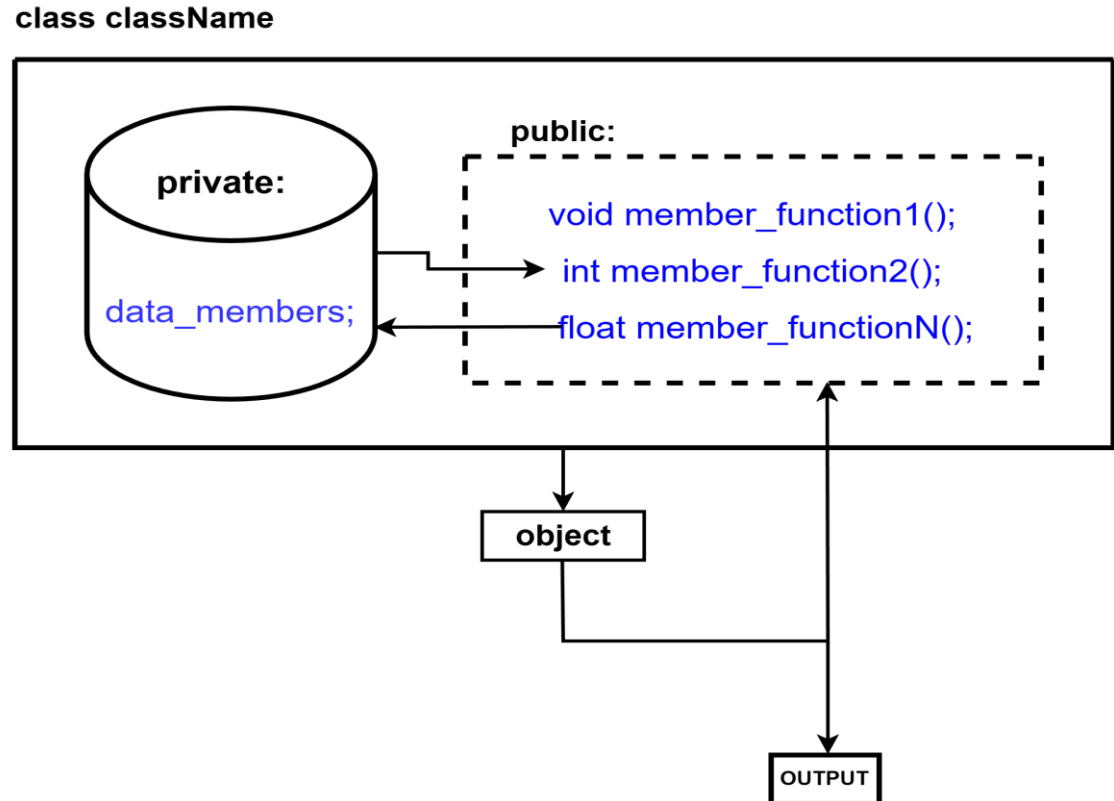
```
class className {  
    private:  
        data_type variableName;  
        data_type variableName2;  
        data_type variableNameN;  
  
    public:  
        data_type memberFunction1();  
        data_type memberFunction2();  
        data_type memberFunctionN();  
};  
  
int main(){  
    className objectVariable;  
    objectVariable.memberFunction1();  
    objectVariable.memberFunction2();  
    objectVariable.memberFunctionN();  
    return 0;  
}  
  
//define member functions within or outside of class  
  
//outside of class example:  
  
void className::memberFunction1()  
{  
    Write your expressions here;  
}
```

```
class student{  
    private:  
        int rollNo;  
        char studName[30];  
        float marks;  
    public:  
        void getData();  
        void showData();  
};  
  
void student::getData()  
{  
    cout<<"Enter roll number:"<<endl;  
    cin>>rollNo;  
    cout<<"Enter student name:"<<endl;  
    cin>>studName;  
    cout<<"Enter marks:"<<endl;  
    cin>>marks;  
}  
  
void student::showData()  
{  
    cout<<"Roll Number"<<"\tStudent Name"<<"\tStudent Marks"<<endl;  
    cout<<rollNo<<"\t\t"<<studName<<"\t\t"<<marks;  
}  
  
int main()  
{  
    student s;  
    s.getData();  
    s.showData();  
    return 0;  
}
```

C++ fundamental concepts

Encapsulation:

- Encapsulation is the bundling of data (**attributes**) and methods (**functions**) that operate on that data within a single unit, typically a class.



Function overloading

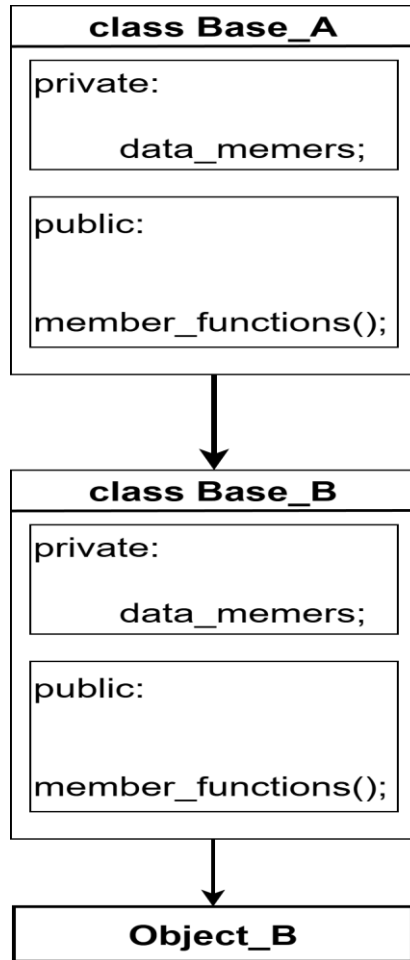
- Function overloading is a **C++** feature that allows a programmer to define multiple functions in the same scope with the same name, provided they have different **parameter lists** (also known as the ***function signature***). This allows functions to perform similar operations on different types or numbers of data, improving code readability and reusability.
- A function's signature determine which specific function is to be called based on the parameters according to is a form of **compile-time polymorphism**. The compiler differentiates functions based on:
 - The **number** of parameters.
 - The **type** of parameters.
 - The **order** of parameters.

Function overloading

- Function overloading/polymorphism is called when functions with same name but with different signature is used to perform different tasks on data. Look at the following different signature of the same name function `add()` , having different parameters.

```
int add(int x, int y);  
int add(int x, float y, int c);  
int add(char firstName, char middleName, char lastName);  
int add(int, int, float z=10.50);
```

Single level inheritance



```
class student{ //base class/parent class
    private:
        int rollNo;
        char studName[30];
        float marks;
    public:
        void getData();
        void showData();
};

class course:public student //derived or child class,
                           //inherited from base class
{
    private:
        ----data_memers;
    public:
        ----member_functions();
};

int main()
{
    course c1; //object of the child class
    c1.getData();
    c2.showData();
    return 0;
}
```

Friend function

- A **friend function** in C++ is a non-member function that is granted special permission to access the private and protected members of a class. It is declared inside the class using the friend keyword but is defined outside the class's scope.
- Friend functions are typically used to:
- Allow external functions, such as utility functions or functions that operate on objects of multiple classes, to access private data when necessary.

Key Characteristics

- **Non-Member:** A friend function is not a member of the class it befriends and does not receive this pointer.
- **Declaration:** It is declared within the class body using the friend keyword. The location within the public, private, or protected section of the class does not affect its access privileges.
- **Definition:** It is defined outside the class without the friend keyword or the scope resolution operator (::), unless it's a member function of another class.
- **Access:** It must be called like a regular function (e.g., functionName(object)) and requires an object (usually passed as an argument) to access that object's specific private or protected members using the dot operator (.).
- **Inheritance:** Friendship is not inherited. A friend of a base class does not automatically become a friend of its derived classes.
- **Transitivity/Reciprocity:** Friendship is not transitive (a friend of a friend is not a friend) nor mutual (if class A is a friend of class B, B is not automatically a friend of A).

friend function

- A non-member function of any class can't access private data members of the class.
- But, declaring a non-member function in side class with keyword **friend**, it allows to access class data members. This function is defined outside of the class independently.

```
class BankAccount {  
private:  
    int balance;  
public:  
    BankAccount(int amount) : balance(amount) {}  
    friend void displayBalance(BankAccount acc);  
};  
void displayBalance(BankAccount acc) {  
    // Can access private member 'balance' using the object 'acc'  
    cout << "Account balance: $" << acc.balance << endl;  
}  
int main()  
{  
    BankAccount myAccount(5000);  
    displayBalance(myAccount); // Called like a normal function  
    return 0;  
}
```


End of Syllabus

Recommended Books:

1. E. Balagurusamy, "Programming in C", Tata McGraw Hill
2. Yashwant Kanetkar, "Let Us C", BPB
3. B. Ram, "Computer Fundamentals", Wiley
4. P.K.Sinha, "Computer Fundamentals".
5. V. Rajaraman, "Fundamentals of Computers", PHI
6. Brian W. Kernigha and Dennis M. Richie: The C Programming Language, PHI
7. Robert Lafore, "Turbo C++"
8. E. Balagurusamy, "Programming in C++", Tata McGraw Hill